

HabloTruck Platform

Complete Use Cases Guide

Stripe, Billing, ManyChat, Company Seats and Timers

Documento completo de use cases del backend actual.

Incluye flujos de negocio, endpoints HTTP, timers y servicios operativos.

Que cubre esta version

1. Alta individual, lifecycle Stripe y access orchestration.
2. Payment Recovery, portal, retry y confirmacion final.
3. Company Seats, invites, joins, over-capacity y healing.
4. Timers, sweepers, reconciliation, retry store y admin functions.

1. Stripe Checkout Handler

Creacion de Checkout Session inicial

Pasos / comportamiento

- StripeCheckoutHandler valida planType, quantity, successUrl y cancelUrl.
- Resuelve el PriceId configurado para individual, fleet/company_seat o cohort.
- Delegates a StripeCheckoutService para crear la Checkout Session.
- El checkout inicial usa card y link; no usa ACH para el alta individual.

Que validar

- PriceId correcto segun plan.
- Checkout URL devuelta correctamente.
- El flujo falla de forma segura si falta plan o precio.

2. Alta Inicial Individual

checkout.session.completed y acceso FULL

Pasos / comportamiento

- StripeWebhookFunction recibe checkout.session.completed.
- StripeSubscriptionHandler.HandleCheckoutCompletedAsync crea o
- actualiza al usuario.
- Persiste StripeCustomerId, StripeSubscriptionId, StripePriceId y
- plan term.
- AccessOrchestrator recompone acceso y sincroniza ManyChat.
- ManyChat puede pasar al usuario a FULL cuando corresponde.

Que validar

- Usuario proyectado correctamente.
- No se activa payment recovery.
- ManyChat no decide la verdad; solo refleja el estado backend.

3. Stripe Subscription Handler

Lifecycle principal driven por webhook

Pasos / comportamiento

- HandleSubscriptionUpdatedAsync aplica reducer pipeline para cambios normales.
- HandleSubscriptionDeletedAsync proyecta cancelacion, paid-through o expiracion final.
- HandleInvoicePaidAsync confirma cobros exitosos y recuperaciones reales.
- HandleInvoicePaymentFailedAsync abre recovery cuando corresponde.
- HandleCustomerUpdatedAsync puede intentar retry automatico del invoice abierto.

Que validar

- Idempotencia por StripeEventId.
- Proteccion contra eventos out-of-order.
- Stripe sigue siendo la source of truth del lifecycle.

4. Access Orchestrator

Recompute centralizado de acceso

Pasos / comportamiento

- Carga el contexto del usuario, seat assignment y entitlement.
- Aplica reglas de dominio para decidir Full, Grace o Blocked.
- Persiste snapshots de acceso efectivo.
- Sincroniza ManyChat best-effort y encola retry si la llamada falla.
- Se invoca despues de eventos Stripe, joins y sweepers.

Que validar

- No hay caminos paralelos de acceso por fuera del orchestrator.
- ManyChat sync es convergente y recuperable.
- Access snapshots quedan consistentes con el dominio.

5. Payment Recovery Journey

Portal de Stripe, customer.updated e invoice.paid

Pasos / comportamiento

- invoice.payment_failed entra por webhook y abre el episodio
- recovery.
- BillingRecoveryManyChatNotifier sincroniza recovery_active hacia
- ManyChat.
- CreateStripePaymentMethodUpdateLinkUseCase genera la portal session
- segura.
- Cuando el usuario actualiza tarjeta, customer.updated puede disparar
- retry automatico.
- RetryStripeOpenInvoiceUseCase sigue existiendo como fallback manual.
- Solo invoice.paid cierra recovery y emite recovered.

Que validar

- No marcar recovered por volver desde Stripe.
- customer.updated no equivale a pago recuperado.
- Retry manual y retry automatico deben converger al mismo final:
- invoice.paid.

6. Create Payment Method Update Link

Endpoint para abrir Customer Portal

Pasos / comportamiento

- Valida actor, ownership y subscriptionId.
- Lee la suscripcion actual desde Stripe.
- Crea una session de Customer Portal en modo payment method update.
- Devuelve URL segura para abrir Stripe.

Que validar

- No cambia lifecycle local por si mismo.
- Debe usarse para recovery, no el payment link inicial.
- Debe validar que el actor sea dueño de la suscripcion.

7. Retry Open Invoice Use Case

Fallback manual para intentar invoice.pay

Pasos / comportamiento

- Valida actor y ownership.
- Obtiene snapshot actual de suscripcion desde Stripe.
- Busca invoice abierta y hace retry cuando aplica.
- Si el usuario sigue en recovery, sincroniza el estado operativo a
- ManyChat.

Que validar

- No marca exito final por si solo.
- Debe seguir esperando invoice.paid como confirmacion real.
- Se usa como fallback o soporte, no como camino principal del
- recovery.

8. Subscription Reminder Service

Auto-renew, save-before-churn y recovery reminders

Pasos / comportamiento

- Corre diariamente desde SubscriptionReminderTimer.
- Lee usuarios con Stripe y determina el journey aplicable segun prioridad.
- Payment Recovery tiene prioridad 1, Save-Before-Churn prioridad 2 y Auto-Renew prioridad 3.
- Deduplica por ventana para evitar doble envio.
- ManyChat recibe solo el reminder vigente para el journey activo.

Que validar

- No deben coexistir reminders contradictorios.
- La deduplicacion es por subscription + window + anchor.
- cancel_at_period_end debe suprimir auto-renew.

9. Cancel At Period End

Cancelacion individual o company al fin de periodo

Pasos / comportamiento

- CancelSubscriptionAtPeriodEndUseCase valida scope individual o company.
- Verifica ownership del actor y que la suscripcion pertenezca al contexto correcto.
- Llama a Stripe para poner cancel_at_period_end=true.
- Luego el webhook y el reducer proyectan el estado local final.

Que validar

- El usuario debe seguir activo hasta CurrentPeriodEndUtc.
- Save-before-churn pasa a ser el journey aplicable.
- No se debe cancelar una suscripcion ajena.

10. Company Seat Checkout

Compra de packs de seats para fleets o schools

Pasos / comportamiento

- Stripe checkout.session.completed no crea usuarios; solo confirma la compra.
- StripeSubscriptionHandler usa metadata companyId/companyName y quantity.
- UpsertFromCheckout crea o actualiza Company.
- Se crea o actualiza Entitlement con SeatsTotal inicial desde Stripe quantity.

Que validar

- SeatsUsed arranca en 0.
- Company y Entitlement deben quedar vinculados al StripeCustomerId.
- No se consumen seats por solo comprar el pack.

11. Invite Management

Create, validate, list y disable invites

Pasos / comportamiento

- CreateInviteFunction crea InviteCode validando que el entitlement exista y este activo.
- GetInviteInfoFunction valida codigo, expiracion, estado y cupo restante.
- ListInvitesForCompanyFunction lista invites por company.
- DisableInviteFunction cambia el invite a disabled.
- JoinCompanyFunction consume el invite solo cuando realmente va a intentar el join.

Que validar

- Invitar no consume un seat.
- Invites pueden expirar, deshabilitarse o agotarse por maxUses.
- La validacion previa debe reflejar el estado real del invite.

12. Company Join Handler

Reserva optimista, idempotencia y acceso company

Pasos / comportamiento

- Carga user, entitlement y seat actual.
- Sincroniza SeatsUsed hacia arriba desde active seats reales.
- Si el seat ya existe y esta activo para ese entitlement, cura la user projection sin volver a reservar.
- Si es un join nuevo: exige entitlement activo, no over-capacity y cupo disponible.
- Reserva capacidad, activa SeatAssignment y recompone acceso.

Que validar

- No sobreasignar seats.
- No reconsumir invite si el join ya habia quedado aplicado parcialmente.
- Bloquear join cuando entitlement no este activo o este over-capacity.

13. Company Seats Sync y Over-Capacity

SeatsTotal continuo y bloqueos por downgrade

Pasos / comportamiento

- SeatsTotal se actualiza desde checkout, subscription.updated,
- subscription.deleted, invoice.paid e invoice.payment_failed.
- Si el webhook viene incompleto, el handler puede enriquecerse con
- snapshot de Stripe.
- Si falta User projection pero existe Company por StripeCustomerId,
- igual proyecta el entitlement.
- Cuando SeatsUsed > SeatsTotal, IsOverCapacity=true y no se borran
- usuarios existentes.
- Nuevos joins quedan bloqueados hasta volver a capacidad valida.

Que validar

- Quantity 0 debe aplicarse como SeatsTotal=0.
- Quantity faltante no debe pisar SeatsTotal existente.
- Over-capacity debe ser estable y entendible para soporte.

14. Entitlement Recount Service

Healing de SeatsUsed e IsOverCapacity

Pasos / comportamiento

- Recuenta SeatsUsed desde SeatAssignments activos.
- Corrige IsOverCapacity en base al recuento real.
- EntitlementRecountTimer lo ejecuta cada hora al minuto 00 UTC.
- Sirve para curar drift por fallos parciales entre reserva y persistencia del seat.

Que validar

- No depende de que expire el entitlement.
- Debe corregir SeatsUsed hacia arriba o hacia abajo.
- Es el background healing principal del modelo Company + Seats.

15. Entitlement Expiry Sweeper

Cierre de entitlements por fin de vigencia

Pasos / comportamiento

- Lee el índice de expiración y procesa entitlements cuyo EndUtc ya venció.
- Marca expired cuando corresponde.
- Borra filas huérfanas del expiry index.
- Dispara recount inmediato para mantener reporting consistente.
- EntitlementExpiryTimer corre cada hora al minuto 10 UTC.

Que validar

- Un entitlement expirado no debe seguir aceptando joins.
- El expiry index debe limpiarse para no reprocesar infinitamente.
- El recount posterior debe dejar SeatsUsed consistente.

16. Grace Sweeper

Cierre de grace individual

Pasos / comportamiento

- GraceSweeperService recorre buckets horarios de grace vencido.
- RecomputeForUserAsync vuelve a evaluar acceso cuando grace termina.
- Limpia el grace index cuando el usuario ya no sigue en grace individual.
- GraceTimer corre cada hora al minuto 00 UTC.

Que validar

- La salida de grace debe quedar reflejada en acceso y ManyChat.
- No debe depender de acciones manuales.
- Debe tolerar fallos por usuario sin detener el sweep completo.

17. Failed Action Retry Service

Reintentos de operaciones ManyChat y reminder dispatches

Pasos / comportamiento

- Lee failed actions vencidas y reintentas según tipo.
- Tiene caps distintos para sync convergente y send-flow user-facing.
- Revalida estado actual antes de reenviar payment recovery o reminders.
- RetryFailedActionsTimer corre cada 5 minutos.
- ListFailedActionsFunction y RequeueFailedActionFunction sirven para
- operación manual.

Que validar

- No debe reenviar mensajes viejos fuera de contexto.
- Retries no deben reactivar journeys cerrados.
- Las acciones non-retryable deben quedar marcadas dead.

18. Stripe Reconciliation Service

Correccion eventual contra Stripe

Pasos / comportamiento

- QueryUsersWithStripeAsync obtiene usuarios con contexto Stripe.
- Lee la suscripcion real desde Stripe y compara contra la proyeccion local.
- Protege contra lecturas stale que intenten revertir estados terminales.
- StripeReconciliationTimer corre diariamente a las 05:00 UTC.
- StripeReplayFunction permite reprocesar eventos ya almacenados.

Que validar

- No debe degradar deleted a active por lecturas viejas.
- Debe recomputar acceso cuando encuentra diferencias reales.
- Es una red de seguridad, no el camino principal del lifecycle.

19. Endpoints de Soporte y Operacion

Funciones auxiliares del backend

Pasos / comportamiento

- HealthFunction devuelve OK para health checks autenticados.
- StripeWebhookFunction es el ingress oficial de eventos Stripe.
- StripePayment_GetPaymentLink expone el alta inicial.
- StripeSubscription_GetPaymentMethodUpdateLink y
- StripeSubscription_RetryOpenInvoice exponen recovery self-service.
- CompanyInviteFunction existe pero hoy es un placeholder trivial; no
- es parte del flujo real de negocio.

Que validar

- La operacion real debe usar los endpoints productivos, no el
- placeholder.
- Webhook, replay y retries forman parte del toolkit operativo
- completo.
- Los contratos HTTP deben mantenerse alineados con ManyChat y
- frontend.

20. QA Integral

Guía corta para pruebas de extremo a extremo

Pasos / comportamiento

- Probar alta individual, renewal, cancel_at_period_end y deleted.
- Probar payment recovery completo: failed -> portal ->
- customer.updated -> invoice.paid.
- Probar company checkout, create invite, get invite info, join
- exitoso y join bloqueado.
- Probar downgrade por debajo del uso y recount hourly.
- Probar replay/retry de eventos y failed actions.

Que validar

- Verificar siempre Stripe IDs, Company, Entitlement, SeatAssignment,
- User y ManyChat.
- recovered solo debe salir con invoice.paid.
- El recount hourly y el retry service deben verse como mecanismos de
- healing, no como source of truth.